ExamForge AI — Metrics Report

Enterprise Metrics Collection System

Generated: 2026-07-25 12:27 UTC

1. Metric Types

Type	Description	Use Cases	Example
Counter	Monotonically increasing value	Request count, error count	api_request_count += 1
Gauge	Point-in-time measurement	Memory, CPU, queue depth, connections	memory_usage = 256MB
Histogram	Distribution of values	Latency, duration, response size	api_latency = 250ms
Ratio	Percentage value	Cache hit ratio, success rate	cache_hit_ratio = 0.85

2. Tracked Metrics

Category	Metric	Type	Unit	Percentiles
API	Endpoint latency	Histogram	ms	avg, p50, p90, p95, p99
AI	Request latency	Histogram	ms	avg, p50, p90, p95, p99
Database	Query latency	Histogram	ms	avg, p50, p90, p95, p99
Sync	Operation duration	Histogram	ms	avg, p50, p90, p95, p99
Exam	Submission time	Histogram	ms	avg, p50, p90, p95, p99
Realtime	Message latency	Histogram	ms	avg, p50, p90, p95, p99
Memory	Usage	Gauge	bytes	Current value only
CPU	Usage percentage	Gauge	percent	Current value only
Battery	Battery level	Gauge	percent	Current value only
Network	Usage	Gauge	bps	Current value only
Cache	Hit ratio	Ratio	percent	Per cache name

3. Latency Tracker Statistics

The LatencyTracker maintains a rolling window of up to 1000 latency samples. It calculates average latency and percentile statistics (p50, p90, p95, p99) from sorted samples. The percentile calculation uses (length * p / 100).floor() as the index into the sorted array. Each tracker is named (e.g., api_/auth/login, ai_generate_questions) and tracks samples independently. The getStats() method returns a map with avg_ms, p50_ms, p90_ms, p95_ms, p99_ms, and sample_count. All latency values are recorded in milliseconds for consistency with web/mobile API conventions.

4. Sampling Rates by Environment

Metric Category	Development	Staging	Production
Crash reports	1.0 (all)	1.0 (all)	1.0 (all)
API latency	1.0 (all)	1.0 (all)	0.5 (50%)
AI latency	1.0 (all)	0.8 (80%)	1.0 (all)
Database latency	1.0 (all)	0.7 (70%)	0.3 (30%)
Sync operations	1.0 (all)	1.0 (all)	1.0 (all)
Exam submissions	1.0 (all)	1.0 (all)	1.0 (all)
UI events	1.0 (all)	0.5 (50%)	0.05 (5%)
Network events	1.0 (all)	0.5 (50%)	0.2 (20%)
Log entries	1.0 (all)	0.8 (80%)	0.3 (30%)
Health checks	1.0 (all)	1.0 (all)	1.0 (all)

5. Performance Overhead Assessment

The metrics system is designed for minimal overhead. All recording operations are O(1) for counters and gauges, O(1) amortized for histogram additions (with O(n) sorting deferred to percentile calculation time). The LatencyTracker stores up to 1000 samples using a Queue, with automatic eviction of oldest samples. Metric entries are buffered (max 500) and evicted on overflow. The getSnapshot() method iterates all trackers and counters but is called only on demand (not on every measurement). Total memory footprint per tracker is approximately 8KB (1000 doubles at 8 bytes each), and 10 trackers would use approximately 80KB — well within the 2% overhead budget for a typical mobile application with 4MB+ memory usage.